Types of Machine Learning

Contents

1. Overview . . . 2
2. Supervised learning . . . 3
   - 2.1 Learning from inputs and outputs . . . 3
   - 2.2 Numerical and categorical data . . . 4
   - 2.3 Regression and classification . . . 4
3. Unsupervised learning . . . 5
   - 3.1 Learning from inputs only . . . 5
   - 3.2 Clustering . . . 6
   - 3.3 Dimensionality reduction . . . 7
   - 3.4 Anomaly detection . . . 9
   - 3.5 Association rule learning . . . 10
4. Semi-supervised learning . . . 11
5. Reinforcement learning . . . 13
   - 5.1 Agent, environment, policy and reward . . . 13
   - 5.2 Example: fire or water . . . 14
   - 5.3 Where RL is used . . . 15
6. Summary . . . 16
7. Sources . . . 17
8. Key terms . . . 17

Where this fits: Pipeline map steps 0 (Foundations), 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Machine learning.
- **Used here, taught in full later:** Labelled data.
- **Leads to:** Applications of ML; Framing an ML problem; Dimensionality reduction; PCA; Simple linear regression; Regression metrics; Multiple linear regression; Logistic regression; Accuracy; Naive Bayes; Decision surface and boundary; K-nearest neighbours; Support vector machines; K-means; Hierarchical clustering; DBSCAN; SayCan: language models choosing skills.

1. Overview

Key point: Based on how much supervision they need, ML algorithms fall into four types: supervised, unsupervised, semi-supervised and **reinforcement learning** (G-1660).

ML algorithms can be grouped in three different ways:

- by **how much supervision** they need (this Note),
- by whether they learn all at once or keep learning over time (batch vs online),
- by how they generalise to new data (instance-based vs model-based).

Supervision (G-1920) means correct answers that guide the algorithm while it learns. Figure 1 shows the four types this produces and their sub-types.

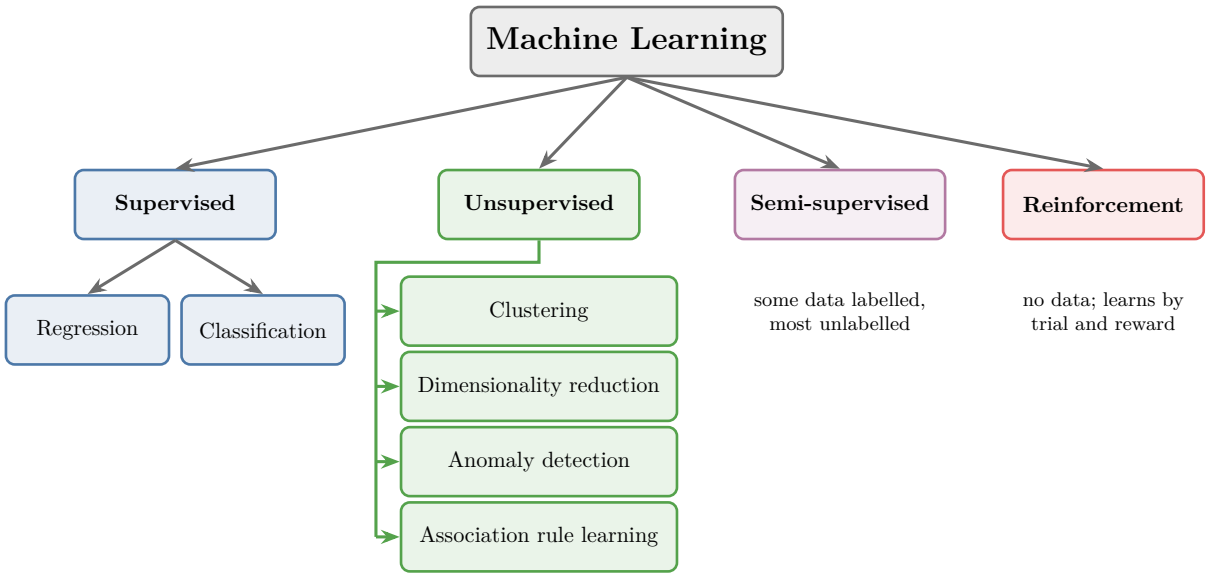


Figure 1: ML types by amount of supervision

2. Supervised learning

> **Key point:** In **supervised learning** (G-1919), the data has both inputs and outputs. The algorithm learns how the output depends on the inputs, then predicts the output for new inputs.

2.1 Learning from inputs and outputs

> **Key point:** Features + target → learn the relationship → predict the target for new observations.

Three words describe any data table:

- a **feature** (G-772) is an input variable (one column of the table);
- the **target** (G-1949) is the output we want to predict (also a column);
- an **observation** (G-1374) is one record (one row of the table).

In **supervised learning** (G-1919), every observation has both its feature values and the correct target value. The algorithm learns the mathematical relationship between them.

Example: campus placements. We have data on 5,000 past students (Figure 2):

- **Features:** IQ and CGPA.
- **Target:** whether the student was placed.
- **Observations:** the 5,000 students, one per row.

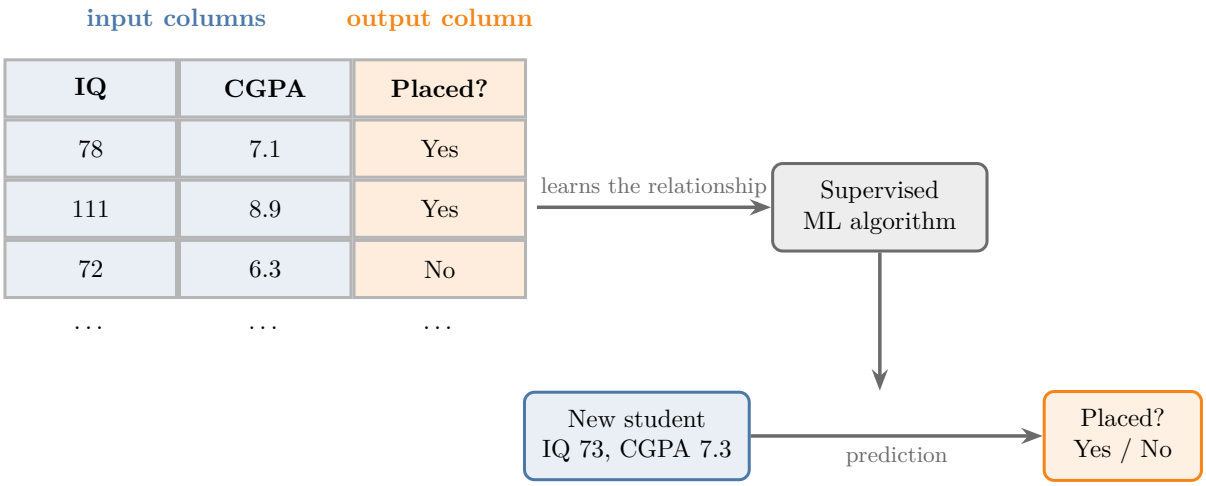


Figure 2: Supervised learning: learn from labelled observations, predict for a new one

After learning, the algorithm can take a new student, say IQ 73 and CGPA 7.3, and predict *yes* or *no*. Most of the ML used in industry is supervised learning.

> **Extra:** The target is also called the **label** (G-1032) or the **output**. Data that includes the target is called **labelled data** (G-1034).

2.2 Numerical and categorical data

> **Key point:** Data is either numerical (numbers) or categorical (categories).

Before splitting **supervised learning** (G-1919) further, we need the two basic types of data:

Type	What it holds	Examples
Numerical	Numbers	Age, weight, height, IQ
Categorical	Categories	Gender, nationality, phone brand

2.3 Regression and classification

> **Key point:** Numerical output: regression. Categorical output: classification.

Supervised learning (G-1919) has two sub-types. Which one we have depends only on the **target**:

- **Regression** (G-1655): the **target** (G-1949) is numerical (**numerical data**, G-1367). *Example:* predicting a student's salary package (4.5 LPA, 3.0 LPA, . . .) from IQ and CGPA.
- **Classification** (G-395): the target is categorical (**categorical data**, G-351). *Example:* predicting placed / not placed from IQ and CGPA.

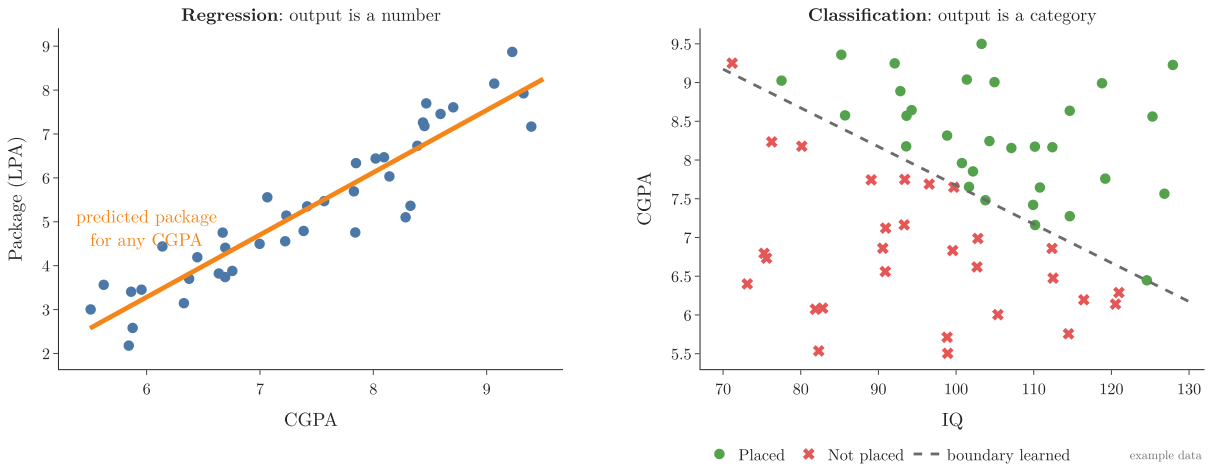


Figure 3: Regression vs classification

In Figure 3, regression fits a line that gives a number for any input. Classification learns a **decision boundary** (G-555): the line or curve that separates the inputs assigned to one category from those assigned to another. If we colour every possible input by the category the model would predict there, we get a **decision surface** (G-560), and the decision boundary is the line where the colour changes.

Problem	Output	Type
Price of a house	A number	Regression
Is this email spam?	Yes / No	Classification
Will it rain today?	Yes / No	Classification
Is there a dog in this image?	Yes / No	Classification
How many dogs are in this image?	A number	Regression

The last two rows have the same input, an image. Only the target decides the type.

3. Unsupervised learning

> **Key point:** In unsupervised learning, the data has features only. There is no **target** (G-1949) to predict, so the algorithm finds structure instead: groups, fewer features, oddities or links.

3.1 Learning from inputs only

> **Key point:** No **target** (G-1949) means no prediction. The algorithm describes the data instead.

In **unsupervised learning** (G-2058), the data has only **features** (G-772). For example, we have the IQ and CGPA of students, but no placement target. Figure 4 shows the same table as Figure 2 with the target column removed.

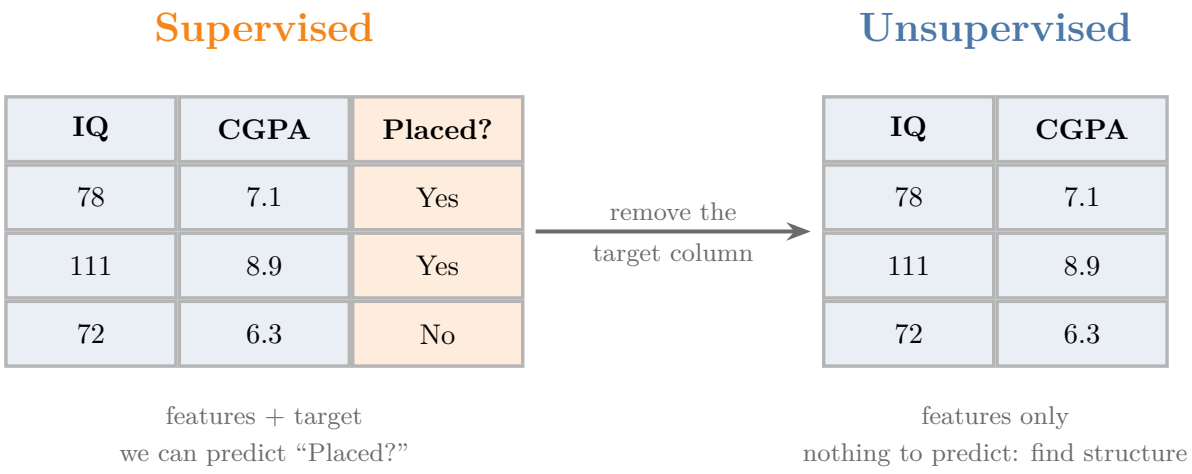


Figure 4: The same table with and without the target column

Figure 5 shows the same change on a scatter plot (a graph where each dot is one student, placed by its IQ on one axis and its CGPA on the other) of 90 example students.

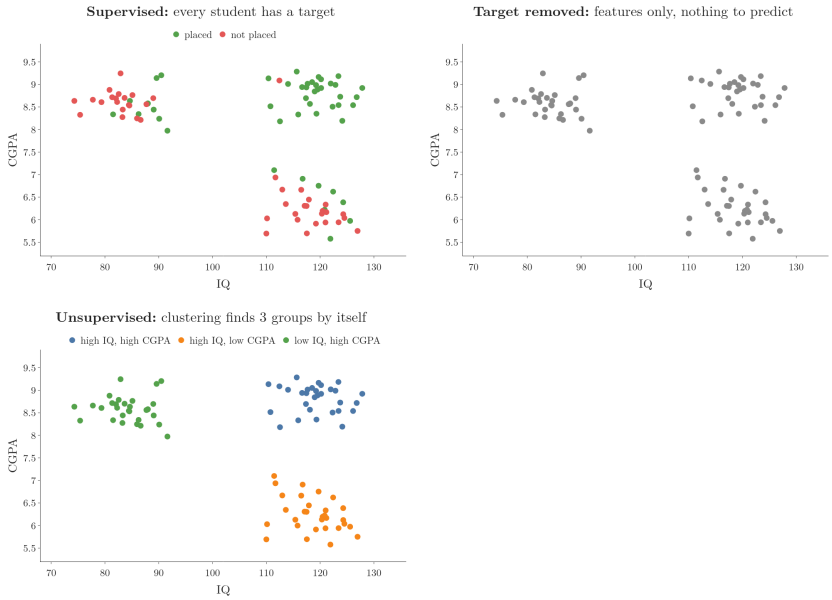


Figure 5: One dataset, seen three ways: with a target (supervised), with the target removed, and after clustering (example data)

1. **Supervised.** Every dot has a colour: placed or not placed. The colour is the target, so an algorithm can learn to predict it.
2. **Target removed.** Every dot is grey. Only IQ and CGPA are left, and there is nothing to predict.
3. **Unsupervised.** A **clustering** (G-401) algorithm (Section 3.2) looks only at IQ and CGPA and colours the three groups it finds. The groups are not "placed" and "not placed": the algorithm never saw that column.

Without a target, we cannot predict anything. Unsupervised learning performs four other jobs:

- clustering (Section 3.2);
- **dimensionality reduction** (G-611) (Section 3.3);
- **anomaly detection** (G-201) (Section 3.4);
- **association rule learning** (G-218) (Section 3.5).

3.2 Clustering

> **Key point:** Clustering finds groups of similar **observations** (G-1374), without being told what the groups are.

Clustering (G-401) splits the data into groups (**clusters**, G-399) of similar observations. Figure 6 shows students plotted by IQ and CGPA; the algorithm found three groups on its own.

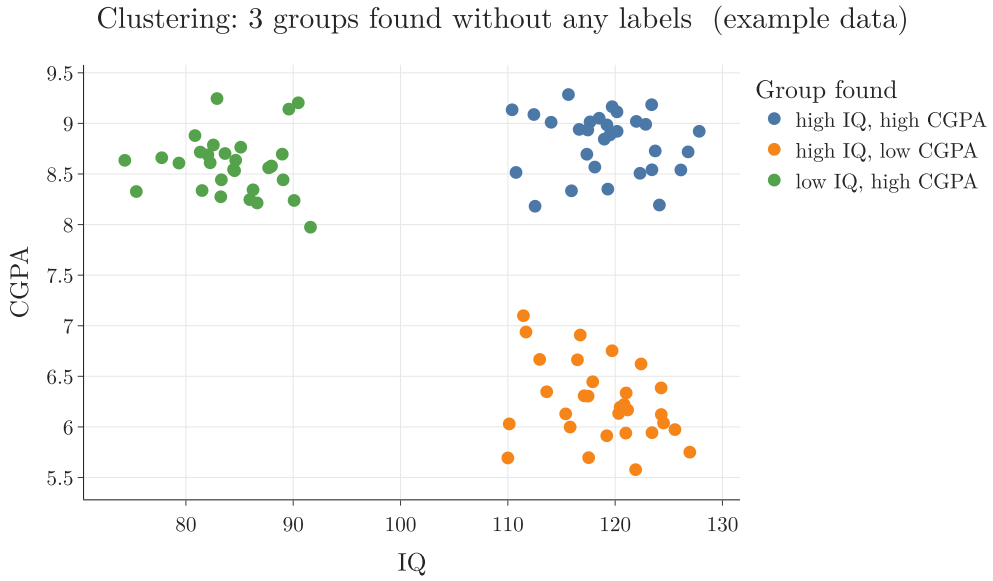


Figure 6: Clustering: groups found without labels

What the groups give us:

- **A category for each student:** for example, high IQ but low CGPA. A new student can be placed in one of the groups.
- **Labels for free:** we can number the groups (0, 1, 2) and then use them as a target for **supervised learning** (G-1919).
- **Customer segments:** an e-commerce site can group its customers by how they behave and treat each group differently.

With two **features** (G-772) we could spot the groups by eye. Clustering also works with hundreds of features, where no human can see the groups.

Hierarchical clustering (G-893; see **hierarchical clustering** (G-893)) goes one step further and finds groups within groups, for example sub-segments inside each customer segment.

3.3 Dimensionality reduction

> **Key point:** Dimensionality reduction cuts down the number of **features** (G-772) while keeping the information. Fewer features speed up learning and let us plot high-dimensional data.

Each feature is a **dimension** (G-610). (A tensor's dimensions mean something else, its number of axes: see rank, axes, shape and size.) Images and text can have thousands of features, which causes two problems:

1. Algorithms become **slow**.

2. After a point, extra features **stop improving** the results, because the same number of **observations** (G-1374) gets spread more and more thinly over the extra dimensions (the curse of dimensionality, see why more dimensions cause trouble).

Dimensionality reduction (G-611) removes the extra features.

Example: house prices. Number of rooms and number of washrooms carry related information. We can combine them into one feature, area (Figure 7).

Rooms	Washrooms	Price
3	2	85 L
2	1	52 L
4	3	120 L

3 columns

combine two related columns into one →

Area (sq ft)	Price
1400	85 L
900	52 L
2000	120 L

2 columns: one fewer, same information

Figure 7: Combining two related features into one

The data has one feature fewer and loses almost no information. Making such a feature by hand, from domain knowledge, is called **feature construction** (G-760; see **feature construction** (G-760)).

When an algorithm such as PCA computes the new features from the data instead, with no domain knowledge, the process is called **feature extraction** (G-762; see **feature extraction** (G-762)).

Visualisation. A graph can show at most 3 dimensions. To see data with hundreds of features, we reduce them to 2 or 3 features and plot those.

Example: handwritten digits. Each digit is an 8 x 8 grid of pixels, which makes 64 features, one per pixel. Figure 8 reduces them to 3 features: images of the same digit land close together.

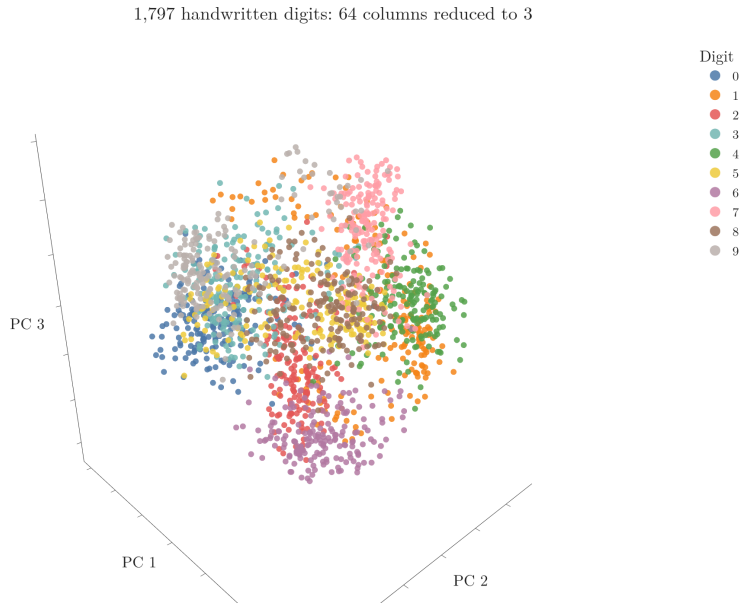


Figure 8: Handwritten digits: 64 features reduced to 3

The technique used for Figure 8 is **PCA** (principal component analysis, G-1469), taught in what PCA is. The Notebook for this Note (`ML-003-types-of-ml.ipynb`) shows Figure 8 as a 3D plot that we can rotate.

> **Extra:** The best-known digits dataset, MNIST, uses 28 x 28 pixel images, which gives 784 features (LeCun et al. 1998). The idea is the same.

3.4 Anomaly detection

> **Key point:** Anomaly detection learns what normal data looks like and flags anything far from it.

Anomaly detection (G-201) finds **observations** (G-1374) that do not fit the pattern of the rest. Typical uses:

- spotting defects in manufacturing,
- catching credit card fraud,
- flagging unusual loan applications.

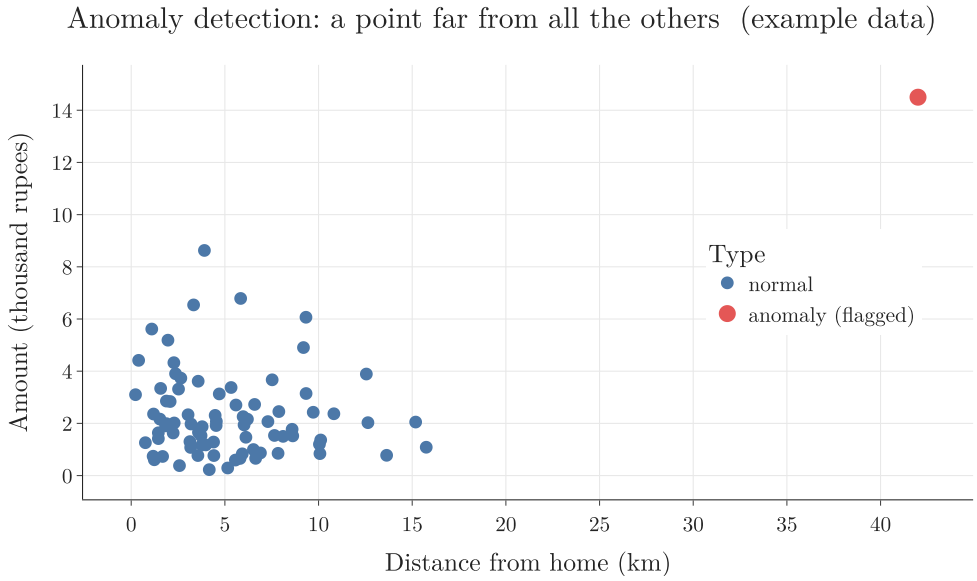


Figure 9: Anomaly detection: one point far from the rest

In Figure 9, almost all transactions are small and close to home. The red one, large and 42 km away, is far from every normal point, so the system flags it. In numbers (the figure's example data): the biggest of the 80 normal transactions is 8.6 thousand rupees and 15.7 km from home, while the red one is 14.5 thousand rupees and 42 km from home. The red one is beyond every normal point on both counts, and 42 km is far outside the 15.7 km that every normal point stays within.

3.5 Association rule learning

> **Key point:** Association rule learning finds items that tend to occur together.

Association rule learning (G-218) looks for "if this, then that" patterns in data. Its classic use is deciding which products a shop places next to each other.

Example: a supermarket. We scan all the bills from the last one or two years (Figure 10). Suppose milk appears in 8 of 100 bills, and eggs appear in 6 of those 8:

Count	Bills
All bills	100
Bills with milk	8
Bills with milk and eggs	6

$$\frac{\text{bills with milk and eggs}}{\text{bills with milk}}$$

$$= \frac{6}{8} = 0.75$$

So 75 percent of the people who buy milk also buy eggs. They usually buy both, so the shop places the two together.

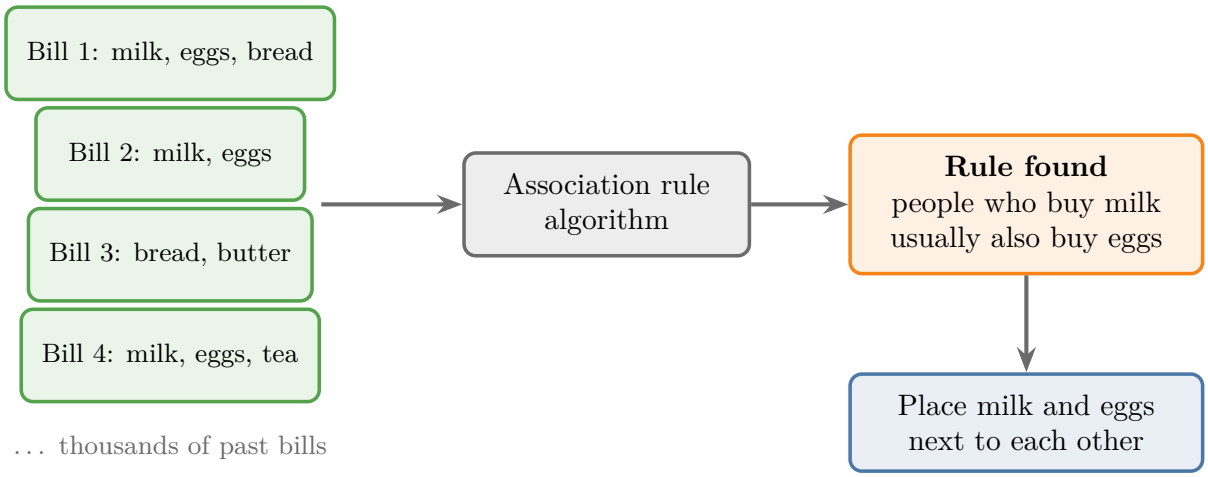


Figure 10: Association rules: from past bills to shelf placement

A famous case: an analysis of shopping baskets at a US store chain found that customers buying baby diapers in the evening often also bought beer (Power 2002). Hidden patterns like this are hard to spot by hand and easy for an algorithm.

4. Semi-supervised learning

> **Key point:** Semi-supervised learning **labels** (G-1032) a few observations by hand and lets the algorithm label the rest.

Labels are **expensive**. Collecting inputs is easy; for example, we can download thousands of images in minutes. But someone has to look at each image and write down what is in it, which costs time and money.

Semi-supervised learning (G-1768) works with data where only a small part is labelled. We label a few **observations** (G-1374), and the algorithm labels the rest automatically.

Example: Google Photos (Figure 11).

1. The app groups photos by the faces in them, without knowing any names. This step is unsupervised.
2. We tag one photo with a name, such as “Dad”.
3. The name spreads to every photo in that group.

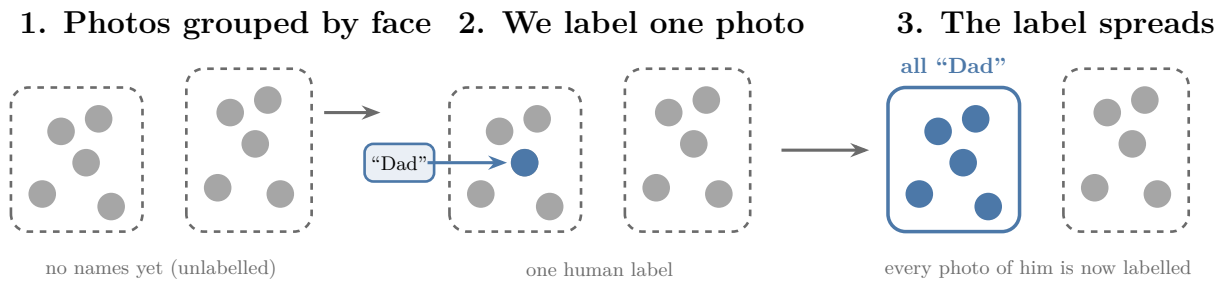


Figure 11: Semi-supervised learning: one label spreads to a whole group

One human label replaced hundreds.

How does one label reach a whole group? Similar observations sit close together, so a label can hop from each observation to its **nearest neighbours** (G-1306), the observations closest to it. Figure 12 shows the hops on the 90 students of Figure 6.

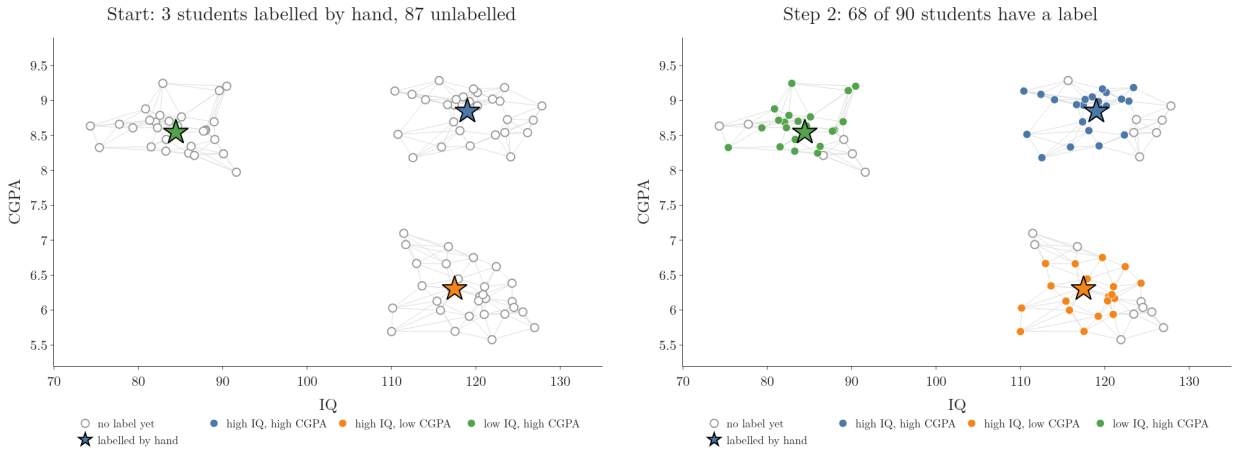


Figure 12: Label propagation on the 90 students of Figure 6. Stars: the three students labelled by hand. Grey links join each student to its 5 nearest neighbours; a white dot has no label yet.

1. **Start.** We label one student in each group by hand (the stars). The other 87 have no label.
2. **Each step.** Every student looks at its linked neighbours, adds up the labels they carry, and takes the strongest one. The three hand labels never change.
3. **The spread.** After step 1, 24 of the 90 students have a label; after step 2, 68; after step 3, 89; after step 4, all 90.
4. **The result.** Every student ends up with the label of its own group, although only three were labelled by hand.

This method is called **label propagation** (G-2172) (Zhu and Ghahramani 2002). It works only when observations with the same label sit close together, as the three groups do here.

A related idea needs no hand labels at all. In **self-supervised learning** (G-2279), the labels come from the data itself: in a sentence, the next word is the label for the words before it (Jurafsky and Martin, §7.7). Language models such as GPT are pre-trained this way; see transfer learning for language.

5. Reinforcement learning

> **Key point:** In **reinforcement learning** (G-1660), there is no ready-made dataset. An agent collects its own experience by acting, gets rewards or punishments, and improves its rules.

5.1 Agent, environment, policy and reward

> **Key point:** The agent acts, the environment rewards or punishes, and the agent updates its policy. Repeat.

Reinforcement learning (RL) (G-1660) starts with no ready-made dataset: nobody hands the algorithm a table of examples with correct answers. The algorithm produces its own data by acting. Each action, and the reward or punishment that follows it, is one piece of experience, and the algorithm learns from this experience, getting better over time (Sutton and Barto 2018, §1.1).

The parts of an RL system:

- **Agent** (G-179): the learner, for example a self-driving car or a game-playing program.
- **Environment** (G-694): the world the agent acts in, for example the road or the game board.
- **Policy** (G-1512): the agent's rule book, saying which action to take in each situation.
- **Reward / punishment** (G-1690): feedback from the environment after each action, good or bad.

Figure 13 shows the loop:

1. The agent observes the environment.
2. Following its policy, the agent takes an action.
3. The environment returns a reward or punishment.
4. The agent updates its policy.

The goal is to collect as much reward and as little punishment as possible.

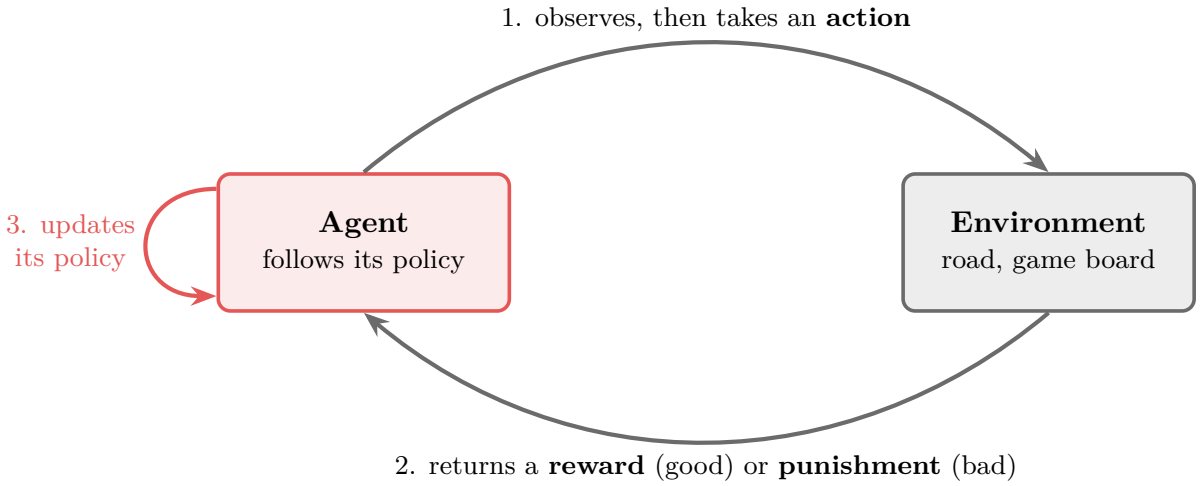


Figure 13: The reinforcement learning loop

5.2 Example: fire or water

> **Key point:** A bad result changes the **policy** (G-1512), so the next choice is better.

Figure 14 shows an **agent** (G-179) that can walk to fire or to water.

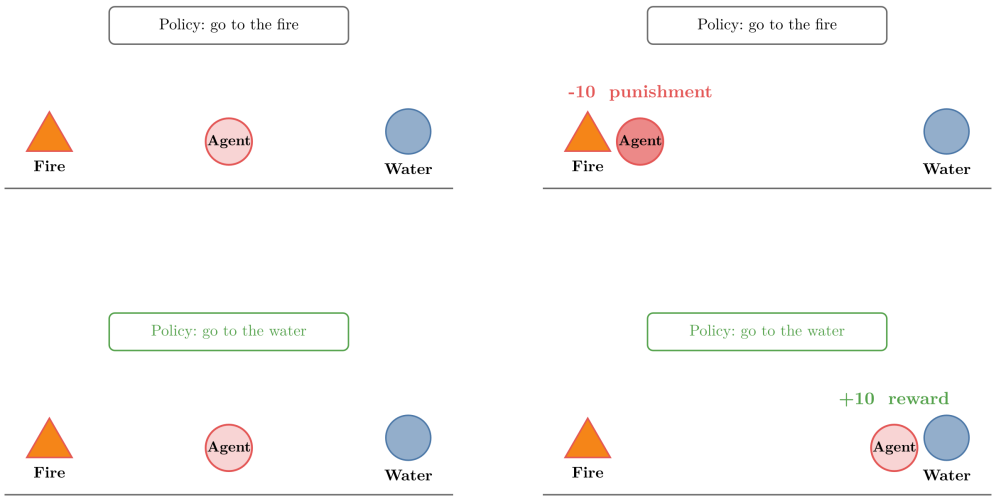


Figure 14: An agent learning to choose water over fire

1. The agent's policy says *go to the fire*. The agent goes, and gets a punishment.
2. The agent updates its policy to *go to the water*.
3. The agent goes to the water and gets a reward.

In numbers, give a punishment the value −10 and a reward the value +10, as in Figure 14 (a choice made for this illustration). The agent's total reward after each trip:

Trip	Place	Reward	Total so far
1	fire	−10	−10
2	water	+10	0
3	water	+10	+10

The policy change after trip 1 is why the total climbs: the agent aims for the highest total.

People learn many things the same way: nobody hands us a dataset for living in a new city; we try, make mistakes and adjust. Training a pet with treats works the same way.

5.3 Where RL is used

> **Key point:** RL beat the world champion at Go and is used for self-driving cars, but it is harder to build than the other types.

- **Self-driving cars:** the car learns to drive by acting on the road and receiving feedback.
- **Games:** Go is far more complex than chess, and beating top humans was thought to be years away. In March 2016, DeepMind's **agent** (G-179) **AlphaGo** beat world champion Lee Sedol in four out of five games (DeepMind 2016).

RL is harder to set up than the other types, but its use is growing fast.

6. Summary

Type	Data	Goal	Example	Why it matters
Supervised: regression	Inputs + numerical output	Predict a number	Salary package from IQ and CGPA	the fitted line gives a number for any input
Supervised: classification	Inputs + categorical output	Predict a category	Placed or not	the decision boundary sorts any new input into a category
Unsupervised: clustering	Inputs only	Find groups	Customer segments	finds groups no human can see in hundreds of features, and gives labels for free
Unsupervised: dimensionality reduction	Inputs only	Fewer features	Rooms + washrooms → area	fewer features speed up learning and let us plot the data
Unsupervised: anomaly detection	Inputs only	Flag unusual observations	Card fraud	a point far from every normal point gets flagged
Unsupervised: association rules	Inputs only	Find items that go together	Milk and eggs	hidden "if this, then that" patterns decide what goes side by side
Semi-supervised	A few labels, many unlabelled observations	Label the rest	Google Photos faces	labels are expensive; one human label replaced hundreds
Reinforcement	No ready-made data; experience and rewards collected from an environment	Learn the best actions	AlphaGo	it learns where nobody can hand over a table of correct answers

- To identify a supervised problem, look at the **target**: number → **regression** (G-1655), category → **classification** (G-395). The input does not decide it: the same image can feed either type.
- No **target** (G-1949) → unsupervised, because without a target there is nothing to

predict, so the algorithm finds structure instead.
- Few **labels** (G-1032) → semi-supervised, because similar **observations** (G-1374) sit close together, so a few hand labels can spread to the rest.
- No ready-made data, only feedback on the **agent** (G-179)'s own actions → reinforcement, because the agent collects its own experience by acting.

7. Sources

Built from

- CampusX, "Types of Machine Learning for Beginners | Types of Machine learning in Hindi | Types of ML in Depth", YouTube, https://www.youtube.com/watch?v=81ymPYEtFOw

Other references

- DeepMind (2016). *AlphaGo*. deepmind.google, research/breakthroughs/alphago.
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft, Chapter 7, §7.7 (self-supervised training). https://web.stanford.edu/~jurafsky/slp3/7.pdf
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE* 86(11).
- Power, D. (2002). What is the "true story" about data mining, beer and diapers? *DSS News*, 10 November 2002.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press. §1.1.
- Zhu, X. and Ghahramani, Z. (2002). Learning from Labeled and Unlabeled Data with Label Propagation. Technical Report CMU-CALD-02-107, Carnegie Mellon University.

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Reinforcement learning (G-1660)	Learning by acting and receiving rewards or punishments.
Supervision (G-1920)	Correct answers that guide an algorithm while it learns.
Supervised learning (G-1919)	Learning from data with inputs and outputs, to predict outputs.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.

Term	Meaning
Label (G-1032)	The answer an example comes with, the value a model learns to predict, such as the digit an image shows; another name for the target or output.
Labelled data (G-1034)	Data that includes the output column.
Regression (G-1655)	Supervised learning with a numerical output.
Numerical data (G-1367)	Data made of numbers.
Classification (G-395)	Predicting which of a fixed set of categories (classes) a row belongs to, learned from examples whose answers are known (supervised learning with a categorical output).
Categorical data (categorical column) (G-351)	Data made of categories: labels rather than numbers.
Unsupervised learning (G-2058)	Learning from inputs only, to find structure.
Clustering (G-401)	Splitting data into groups of similar rows.
Anomaly detection (G-201)	Finding rows that do not fit the pattern of the rest.
Association rule learning (G-218)	Finding items that tend to occur together.
Cluster (G-399)	One group found by clustering.
Dimension (G-610)	One feature (input column) of the data, seen as one axis of the space the observations live in: 10 features give 10 dimensions. A tensor's dimensions mean something else, its number of axes.
Semi-supervised learning (G-1768)	Learning from a few labelled rows and many unlabelled ones: the algorithm labels the rest from the few labels, which saves the cost of labelling everything by hand.
Label propagation (G-2172)	Semi-supervised method in which labels hop from labelled observations to their nearest neighbours, step by step.
Agent (G-179)	The learner in reinforcement learning, such as a self-driving car or a game-playing program: it acts in an environment, gets rewards or punishments, and improves its rules for acting.
Environment (G-694)	In reinforcement learning, the world the agent acts in, such as the road or a game board; it rewards or punishes each action, and the agent learns from that.
Policy (G-1512)	The agent's rules for which action to take.
Reward / punishment (G-1690)	Good / bad feedback after an action.

Term	Meaning
Input / output (G-948)	The columns we know / the column we want to predict.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Decision surface (G-560)	A plot colouring every point of the input space by the class the model would predict there.
Dimensionality reduction (G-611)	Reducing the number of input columns while keeping the information.
Hierarchical clustering (G-893)	Clustering that builds a hierarchy of clusters, from single points up to one cluster.
Feature construction (G-760)	Creating a new column by hand from existing ones, e.g. rooms + washrooms into area.
Feature extraction (G-762)	Letting an algorithm such as PCA produce new columns from the existing ones (compare feature construction, where we make them by hand).
PCA (principal component analysis) (G-1469)	A way to replace many columns with a few new ones that keep most of the spread in the data, using no labels (unsupervised feature extraction for dimensionality reduction). Each new column, a principal component, follows one direction of greatest variance. Used to cut the number of features and to plot data with many columns.
Nearest neighbours (neighbours) (G-1306)	The observations at the smallest distance from a given point; in KNN, the k training points closest to the query point, whose values or labels decide the prediction.
Self-supervised learning (G-2279)	Learning where the labels are made from the data itself, such as the next word of a text; it lets a model learn from any amount of unlabelled data, with no human labelling.